

When Does Population-Mobility OD Injection Help Traffic Forecasting? A Capacity-Gap Account and a Validation-Gated Remedy

Sumin Han

Abstract—Road traffic is a downstream consequence of population movement, so a forecast of origin–destination (OD) flow, routed onto the road network, should tell a traffic model something its own history does not. We test this on Seoul (500 OD zones; 121 volume and 396 speed sensors; 2023–2026) under 30-fold rolling-origin validation, and find that *how* the signal is injected matters more than whether it carries information. Injected uniformly through a learned gate, routed OD reduces a lightweight graph-recurrent model’s MSE by 7.2% (volume) and 8.2% (speed), but across ten published architectures (31k–774k parameters) the same mechanism helps only the two smallest and is significantly *harmful*, on at least one task, for every architecture above 10^5 parameters – a capacity-gap pattern (Spearman $\rho = 0.89$ on volume) that a within-architecture capacity sweep also reproduces. A simple remedy removes this limitation: a per-sensor selector, fitted on each fold’s validation split alone and requiring no retraining, chooses between the plain and injected checkpoint for every sensor. Selective injection improves all eleven models in absolute RMSE, reaches significance in eighteen of twenty architecture/task combinations – including eight of the ten where uniform injection was significantly harmful. We also report the checks that bound these claims: most of the uniform gain on the lightweight model is explained by a calendar climatology of the signal (the calendar-free residual alone recovers 34%), and a seed-control experiment shows that, net of selection noise, the selector’s gain stays significant in eleven of twenty combinations and reverses in the three that uniform injection harmed most.

Index Terms—Traffic prediction, origin–destination flow, road network routing, spatio-temporal knowledge injection, urban computing, empirical study, graph neural networks.



1 INTRODUCTION

Short-term forecasts of road traffic volume and speed underpin congestion management, route guidance, and signal control. The dominant modeling paradigm treats this as a purely *endogenous* problem: a deep spatiotemporal model observes past sensor readings on a road-network graph and extrapolates them, using graph convolution [1]–[3], attention [4]–[6], or learned graph structure [7]–[9]. This framing ignores information that is causally *upstream* of traffic: the origin–destination (OD) movement of the population that produces it. A commuter’s decision to travel precedes, by minutes to hours, the vehicle that decision puts on a specific sequence of road links. If population movement can be forecast, and correctly attributed to the links it will cross, it should give a traffic model information its own history does not contain.

The question is practically relevant. Transport authorities increasingly have access to aggregate population-mobility products (mobile-carrier location records, card-transaction geodata) at cadences fast enough to be predictive. Injecting such a signal into an already deployed model is far cheaper than re-architecting around a larger one – *if* it reliably helps. Knowing when it helps, and when it does not, is therefore as valuable to a practitioner as a single positive result.

Prior work that brings regional context into road-level prediction does so through a *learned*, implicit mapping – e.g., [10] attends from region-level embeddings (LTE traces, POIs, imagery) to road-level representations on Seoul speed data. We test an *explicit* channel instead: a road-hierarchy-weighted shortest-path assignment routes the forecast OD flow onto the links it would physically cross, so the signal injected at a sensor has a concrete, checkable justification.

We evaluate this with 30-fold rolling-origin validation (12-month train / 1-month validation / 1-month test, 1-month stride), paired significance tests on every comparison, and two sensor modalities. The study unfolds in three steps, each answering the question the previous one raises.

Step 1 – a lightweight model benefits. Injecting the routed OD forecast through a learned per-node, per-step gate reduces our lightweight graph-recurrent model’s MSE by 7.2% (volume, 29/30 folds) and 8.2% (speed, 30/30), both $p < 10^{-7}$. Controls show the road-network routing is essential (an unrouted signal does nothing) and that much of the gain is calendar-shaped, with a real but smaller share (34%) carried by day-specific OD content.

Step 2 – uniform injection does not generalize. Applied to ten published architectures spanning 31k–774k parameters – STID [11], GMAN [5], STGCN [1], PDFormer [12], STAEformer [6], MTGNN [9], DCRNN [2], GTS [8], Graph WaveNet [3] and AGCRN [7] – the same mechanism significantly helps only the two smallest (STID and GMAN, on volume), and is significantly harmful on at least one task for every architecture above 10^5 parameters. The effect tracks model size (Spearman $\rho = 0.89$ on volume), and scaling a

- Sumin Han is with the Korea Science Academy of KAIST, Busan, Republic of Korea (e-mail: suminhan@ksa.kaist.ac.kr).
- Code, analysis scripts, and result summaries: <https://github.com/SuminHan/traffic-od-injection-capacity-gap>

single architecture (STID) reproduces the same decay and sign reversal. An oracle experiment rules out OD forecast error as the cause. We call this the *capacity gap*: once a model is expressive enough to infer what the signal says, a fixed fusion mechanism cannot extract net value from it.

Step 3 – selective injection recovers the benefit. The capacity gap limits what an architecture’s *own* fusion mechanism can learn, not whether the signal carries value. A per-sensor selector, fitted on each fold’s validation split alone, chooses between the plain and the injected checkpoint for every sensor, with no retraining. It improves all eleven models in absolute RMSE, is significant in eighteen of twenty architecture/task combinations – including eight of the ten where uniform injection was significantly harmful. A random-selection control and a seed-control experiment test whether this gain is real or an artifact of selection noise: net of the measured noise baseline it remains significantly favorable in eleven of twenty combinations, and the three combinations where it reverses are exactly the three uniform injection harmed most.

Our contributions are:

- **A practical, architecture-agnostic recipe.** Selective, validation-gated injection recovers auxiliary-signal value that a fixed fusion mechanism fails to extract, with no architectural change and no training beyond the two checkpoints the comparison already needs. Its gains are validated against random-selection and seed-noise controls, and we report where they do not survive.
- **A systematic negative result and its explanation.** Routed OD injection significantly improves a lightweight model, but the same mechanism fails to generalize across ten published architectures in a way that is ordered by model capacity. An oracle experiment, a per-sample diagnostic, and two capacity sweeps (within our model and within STID) support a capacity-gap account over the alternatives of forecast error or insufficient fusion expressiveness.
- **A reproducible study design.** Road-network routing of an OD forecast into a sensor-level signal, 30-fold rolling-origin evaluation throughout, calendar-confounding and routing controls, and reimplement-fidelity checks against published benchmark numbers; code and per-fold results are public.

Section 2 reviews related work, Sections 3–5 define the problem, method and setup, Section 6 presents Step 1, Section 7 presents Steps 2 and 3, Section 8 discusses limitations, and Section 9 concludes. The supplementary material contains a fusion-architecture ablation, an adaptive-threshold refinement of the selector, cross-dataset replication, a qualitative case study, two exploratory findings, and the reimplement-fidelity check.

2 RELATED WORK

2.1 Traffic Forecasting Models

Classical approaches – ARIMA [13] and Kalman filtering [14] – model each sensor as a low-dimensional linear

time series and cannot capture the nonlinear spatial propagation of congestion; deep spatiotemporal models have repeatedly been shown to dominate them on road-sensor benchmarks [2], [3], so we do not re-run them here. The now-dominant paradigm combines graph-structured spatial mixing with temporal sequence modeling. STGCN [1] pairs graph convolution with gated temporal convolution; DCRNN [2] models diffusion on a directed graph inside a recurrent encoder-decoder; ASTGCN and ASTGNN [4], [15] add spatial and temporal attention; STSGCN [16] models local spatiotemporal correlation jointly; T-GCN [17] and ST-ResNet [18] combine graph or convolutional spatial modeling with recurrent or residual temporal modeling. A second family learns the graph itself: Graph WaveNet [3] learns a dense adaptive adjacency alongside dilated causal convolutions, AGCRN [7] learns node-specific parameters and adjacency from node embeddings, GTS [8] samples a discrete structure via Gumbel-softmax, and MTGNN [9] learns a graph for arbitrary multivariate series. Attention-based and efficiency-oriented designs include GMAN [5] (stacked spatial/temporal attention), D2STGNN [19], PDFormer [12] (explicit propagation delay), STAEformer [6] (adaptive embeddings on a vanilla Transformer), STID [11] (identity embeddings with an MLP backbone) and BigST [20]. We use ten of these – STID, GMAN, STGCN, PDFormer, STAEformer, MTGNN, DCRNN, GTS, Graph WaveNet and AGCRN – as the published architectures of Section 7, chosen to span a $25\times$ range of parameter counts and qualitatively different treatments of spatial structure (fixed diffusion, learned adjacency, learned discrete structure, attention, and graph-free embeddings).

2.2 Population/Regional-Knowledge-Aware Traffic Prediction

Prior work has explored injecting exogenous population or regional context into road-level traffic models. Most directly related is [10], discussed above, which embeds real-time region-level knowledge via a *learned* bipartite spatial transform attention to convert region-level to road-level representations, evaluated on Seoul traffic speed and living-population data in three downtown districts. Our approach differs in kind, not just degree: instead of a learned implicit region-to-road mapping, we compute an *explicit* shortest-path assignment over the actual road network, so a given sensor only receives signal from OD pairs whose routed path provably crosses that sensor’s link. Our own prior work [21] established the foundational floating-population embedding this line of work builds on, applied to traffic prediction without explicit road-network routing.

2.3 Knowledge Adaptation Between Data-Rich and Data-Sparse Sources

Beyond traffic specifically, knowledge adaptation from a data-rich source to a data-sparse target is a recurring pattern in urban prediction. Within a single city, [22] transfers demand-forecasting knowledge from a station-intensive public-transit mode (bus) to station-sparse modes (train, light rail, ferry) via a memory-augmented multi-task network with a learned *boost/eliminate* dual-vector gate, reporting that naive feature concatenation of the two sources

actively *hurts* accuracy relative to the target-only model – a finding we independently reproduce in a different setting in Appendix A (supplementary material). We adapt this dual-gate mechanism as one of the injection mechanisms tested in Section 7. Across cities, RegionTrans [23] matches regions between cities to transfer spatiotemporal prediction knowledge from a data-rich source city to a data-scarce target city; CrossTReS [24] selectively re-weights source-city regions by relevance before transfer; and, in a different urban sensing domain, FLORAL [25] adapts semantically related dictionaries across modalities for air-quality prediction. Across this broader literature, reported successes share a consistent structural signature: the target is a genuinely data- or capacity-limited learner – few stations, a short training history, a data-poor city, or, in our terms, a lightweight model. This pattern is consistent with, and directly motivates, the capacity-gap explanation we develop in Section 7.

2.4 Foundation and Universal Models for Spatiotemporal Prediction

A separate and rapidly growing line of work pursues generalization *across* datasets, cities, and even modalities rather than richer per-dataset signal. UniST [26] pre-trains a single Transformer across diverse spatiotemporal domains via masked-token modeling and adapts to new scenarios with knowledge-guided prompts in few-/zero-shot settings. GPT-ST [27] and STEP [28] similarly pursue generative or predictive pre-training for spatiotemporal graphs. Most recently, large language models have been adapted directly to spatiotemporal forecasting: UrbanGPT [29] couples a spatiotemporal dependency encoder with an instruction-tuned LLM for zero-shot urban prediction; ST-LLM [30] tokenizes per-location time steps with spatial-temporal embeddings and partially freezes LLM attention to adapt it to traffic data; and OpenCity [31] combines Transformer and graph-neural-network components into an open foundation model with promising zero-shot and scaling behavior across cities. This direction is orthogonal to ours – these works study generalization of a shared backbone across datasets and cities, whereas we study a single-city, cross-modality knowledge-injection mechanism and characterize precisely when and why it helps, which is a question that remains unanswered regardless of how the underlying backbone is pre-trained or shared.

3 PRELIMINARIES AND PROBLEM DEFINITION

Road network. The capital-region road network is represented as a directed graph $\mathcal{G} = (\mathcal{V}_r, \mathcal{E}_r)$ with 315,048 nodes and 430,675 directed edges, built from MOCT LINK/NODE shapefiles restricted to the capital-region bounding box. Each edge $e = (u, v)$ has weight

$$w_e = \ell_e \cdot \rho_e^\alpha, \quad (1)$$

where ℓ_e is physical edge length in meters, $\rho_e \in (0, 1]$ is a road-hierarchy discount factor mapping the seven MOCT road-rank categories to values from ≈ 0.50 (local road) up to 1.00 (motorway), and $\alpha = 0.9$ controls the strength of

the hierarchy preference relative to raw distance in shortest-path routing (higher α more strongly prefers major roads even at the cost of extra distance).

OD movement. $\mathcal{X}_{od} \in \mathbb{R}^{T \times N \times N}$ is the hourly origin–destination population flow between $N=500$ administrative (dong-level and, outside Seoul proper, gu/si-level) units for T hours spanning 2023-01 through 2026-07/08, derived from mobile-phone location records and released as a public living-mobility dataset.

Traffic sensors. Two independent sensor sets from Seoul’s TOPIS system: $S_v=121$ point volume sensors (vehicles/hour) and $S_s=396$ link-level speed sensors (km/h), each snapped to the nearest road-graph edge by minimizing Euclidean distance in a projected coordinate system (mean snap distance 46m and 25m respectively, reflecting that speed sensors already carry native link geometry while volume sensors are point counters).

Forecasting task. Given $P=12$ hours of history for every sensor, forecast the next $Q=12$ hours: $\hat{X}_{t+1:t+Q} = \mathcal{F}_\theta(X_{t-P+1:t})$. We report MSE, RMSE, MAE, and MAPE, with a near-zero-denominator guard on MAPE (≥ 10 vehicles/h for volume, ≥ 1 km/h for speed) chosen per-modality because near-zero speed is a genuine congestion state that should not be treated as a sparsity artifact, whereas near-zero volume typically is.

Rolling-origin evaluation. All results use walk-forward validation: 12 months train / 1 month validation / 1 month test, stride 1 month. The sensors’ native 2023–2025 range gives 23 folds; re-aggregating raw monthly files that an upstream script had silently stopped ingesting extends this to 30 folds over 2023–2026.07. Every headline comparison in this paper (Tables 4–11) uses the full 30-fold range. A few secondary diagnostics were run before the extension and keep their original fold counts (the fold-0 capacity sweep of Section 6, the oracle and mechanism (ii)–(v) pilots and the per-sample diagnostic of Section 7); each states its count, and folds from different ranges are never mixed within one significance test.

The injection question. Let $\mathcal{F}_\theta$ be a forecaster taking only past traffic, and $\mathcal{F}'_{\theta, \phi}$ the same architecture augmented with a routed OD signal $od_{t-P+1:t+Q}$ through a fusion module with parameters ϕ (Section 4.3). With enough capacity, ϕ can always be fit to reduce *training* loss, or to ignore an unhelpful signal. The question is whether $\mathcal{F}'$ generalizes significantly better than $\mathcal{F}$ on held-out test months under a paired, walk-forward evaluation – and whether the answer depends on the capacity of $\mathcal{F}_\theta$, which is why we test the same signal across eleven models of differing size.

4 METHODOLOGY

4.1 OD Movement Forecasting

The OD forecaster (*HybridOD*) predicts each administrative unit’s hourly outflow one step ahead. Its graph connects the $N=500$ units by their historical (training-period) OD flow, so two distant districts joined by a fast subway line are strongly connected while adjacent but poorly connected neighborhoods are not. A graph GRU propagates information over this graph with edge weights modulated by the current hour’s calendar features (hour-of-day, day-of-week, and Korean public-holiday indicators for the day and

TABLE 1
Notation used throughout the paper.

Symbol	Meaning
$\mathcal{G}, \mathcal{V}_r, \mathcal{E}_r$	Road network graph, nodes, directed edges
$w_e, \ell_e, \rho_e, \alpha$	Edge weight, length, hierarchy discount, its exponent
$\mathcal{X}_{od}$	Hourly $N \times N$ OD movement tensor
N	Number of OD administrative units (500)
S, S_v, S_s	Sensor count (general, volume = 121, speed = 396)
P, Q	Input / forecast horizon length (12, 12 hours)
W	Static $500 \times S$ routing-weight matrix
$\text{share}(o \rightarrow d)$	Historical destination-share distribution of origin o
od_t, routed_t	Per-origin outflow forecast / routed sensor signal at hour t
$h_t^{\text{traffic}}, h_t^{\text{od}}$	Traffic-branch / OD-branch hidden state at step t
$\beta_t, g(\cdot)$	Per-node/per-step fusion gate, its (MLP) parameterization
θ, ϕ	Base-model / fusion-module parameters

its neighbors), and its hidden state is fused, through the same per-node gate used downstream (Section 4.3), with a calendar-only branch that captures each unit’s typical outflow. This design was selected by an architecture search over 14 rolling-origin folds, in which both a pure graph GRU and a pure calendar baseline underperformed the fused model. HybridOD is frozen at a single fold-0 checkpoint, so every downstream experiment receives an identical OD forecast and upstream forecast quality cannot vary across folds.

4.2 Road-Network Routing

For each origin unit o , Dijkstra’s algorithm on $\mathcal{G}$ (edge weights as in Section 3) gives a shortest-path tree to every other unit; this is run once per origin (not per OD pair), making the full sweep over all $N=500$ origins tractable (≈ 120 seconds on the full 315k-node graph). Each origin’s historical (2023 training-period) destination-share distribution $\text{share}(o \rightarrow d)$ is computed from the ground-truth OD tensor. A static routing-weight matrix $W \in \mathbb{R}^{500 \times S}$ is built once:

$$W[o, s] = \sum_d \text{share}(o \rightarrow d) \cdot \mathbb{1}[\text{path}(o, d) \text{ crosses sensor } s\text{'s link}] \quad (2)$$

Applying W to any hour’s per-origin outflow forecast via a single matrix multiplication $\text{routed}_t = \text{outflow}_t \cdot W$ yields a routed sensor-level signal for that hour. W is a purely structural object – built once from the road network and the 2023 training-period destination-share distribution – and is reused unchanged across every rolling-origin fold, since it encodes road-network topology and typical destination preference rather than any particular hour’s traffic. This routed signal, not a unit’s raw outflow pasted onto its nearest sensor, is what we inject: a raw outflow says nothing about *which* roads the trips use, and Section 6.1 confirms quantitatively that the unrouted signal carries no benefit.

Relation to classical traffic assignment. Equation (2) is, in the terminology of the transportation literature, a deterministic *all-or-nothing* network loading: each origin’s demand is assigned entirely to shortest paths, with no

congestion feedback and no dispersion in route choice. It therefore satisfies neither of Wardrop’s equilibrium principles [32] and does not solve the convex program that makes user equilibrium computable [33], [34]. This is deliberate, and we state the tradeoff plainly because a transportation-side reader will notice it immediately. An equilibrium assignment would require link performance functions, a congestion feedback loop, and an iterative solve for every one of the 31,392 hours in the study range, and – more fundamentally – would make the injected signal a function of the very congestion state the downstream model is trying to predict, reintroducing exactly the circularity that a forward-looking demand proxy is meant to avoid. What we need from W is not an accurate flow prediction but a stable structural map from origins to the sensors their outflow plausibly traverses; the learned gate of Section 4.3 is what decides how much to trust the resulting signal, hour by hour and sensor by sensor.

We also tried a stochastic multi-path variant of W (five Dijkstra passes per origin with log-normal edge-weight noise, crediting sensors fractionally), a crude stand-in for Dial’s logit assignment [35]. It performed no better than the single-shortest-path W , suggesting that route-choice dispersion is not the binding limitation at this spatial and temporal granularity.

4.3 Gated Fusion Injection (Primary Mechanism)

The traffic model is a graph-recurrent encoder-decoder over a k -nearest-neighbor, Gaussian-kernel-weighted sensor graph (*WeightedGraphGRUCell*), with the kernel bandwidth set adaptively to the median nearest-neighbor distance among sensors. At every decode step t , the routed OD signal is projected through a small MLP, $h_t^{\text{od}} = \text{MLP}(od_t)$, and fused with the traffic branch’s hidden state via a per-node, per-timestep learned gate:

$$\beta_t = \sigma\left(g([h_t^{\text{traffic}}; h_t^{\text{od}}])\right), \quad (3)$$

$$h_t^{\text{final}} = \beta_t h_t^{\text{traffic}} + (1 - \beta_t) h_t^{\text{od}}, \quad (4)$$

with the recurrence carried forward at every step by the *pure* traffic branch h_t^{traffic} (not the fused state h_t^{final}), so that the OD signal can modulate the current step’s prediction without permanently altering the model’s internal traffic-only state trajectory – following the design validated for the OD forecasting model itself. A single global scalar gate (one β shared across all nodes and timesteps within a batch) was tried first and found strictly worse than this per-node/per-step design, confirming that the model needs to learn context-dependent trust in the auxiliary signal rather than a single fixed blending weight.

5 EXPERIMENTAL SETUP

Datasets. TOPIS traffic volume (121 sensors) and speed (396 sensors), and Seoul capital-region OD movement (500 administrative units), 2023-01 through 2026-07/08, as described in Section 3. Table 2 summarizes basic statistics.

Models. Our own lightweight model (Section 4.3; hidden size 48, 62k parameters) and the ten published architectures of Section 2.1, each reimplemented at the scale of our sensor networks with a shared recipe: $P=Q=12$, Adam, learning

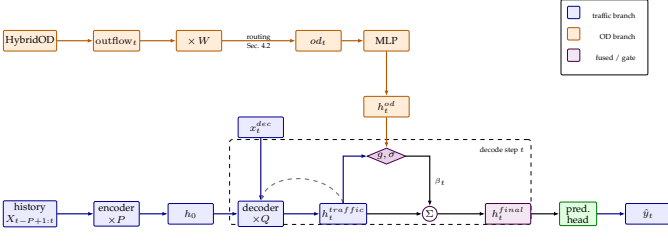


Fig. 1. End-to-end gated-fusion injection architecture (Section 4.3). *Upper left*: the frozen HybridOD forecast is routed onto the road network via the static matrix W (Section 4.2) to produce od_t . *Main flow*: the traffic encoder-decoder consumes recent history; at each decode step the routed signal is projected to h_t^{od} and combined with the traffic branch's $h_t^{traffic}$ through a per-node, per-timestep gate β_t (equation in text). Only the pure $h_t^{traffic}$ (not the fused h_t^{final}) is carried into the next step's recurrence.

TABLE 2
Dataset summary.

	Volume	Speed
Sensors	121	396
Unit	vehicles/hour	km/h
Sensor type	point counter	link geometry
Mean road-graph snap dist.	46 m	25 m
Date range	2023-01 – 2026-07	
OD administrative units	500	

rate 2×10^{-3} , up to 40 epochs with early stopping (patience 8) on validation loss, and smooth- L_1 loss (Table 3). Two reimplementations matter for the results. GMAN's pure self-attention has no sequential inductive bias and failed to train until we added sinusoidal positional encoding over the lookback axis. A PEMS-BAY fidelity check against published numbers (supplementary Appendix F) shows DCRNN, Graph WaveNet and STAEformer reproducing published accuracy within normal reimplementation variance, and exposed a bug in our first STID, which had replaced its published discrete time-of-day/day-of-week embeddings with continuous features; every STID result reported here uses the corrected architecture. Each architecture is trained in a plain (no-OD) form and with OD injection (Section 7).

Evaluation and hardware. We report MSE, RMSE, MAE and MAPE per fold (Section 3) and compare configurations with a paired t -test and a Wilcoxon signed-rank test over the per-fold metric pairs. Adjacent folds share 11 of 12 training months, so folds are not fully independent and p -values should be read as indicative (Section 8). All experiments run on a single NVIDIA H200 GPU. Code, exact fold boundaries, and per-fold results for every reported number are public (footnote, page 1); raw data and checkpoints are omitted for size, but the repository documents each data source.

6 MAIN RESULT: INJECTION ON A LIGHTWEIGHT MODEL

Both tasks reach paired- t $p < 10^{-5}$ on MSE, RMSE and MAE (speed: $p < 10^{-13}$ on all four metrics), and the Wilcoxon signed-rank test agrees in every case. The volume gain is not driven by a few large wins: 29 of 30 folds improve, and the single regression (fold 23, test month 2026-01) is small

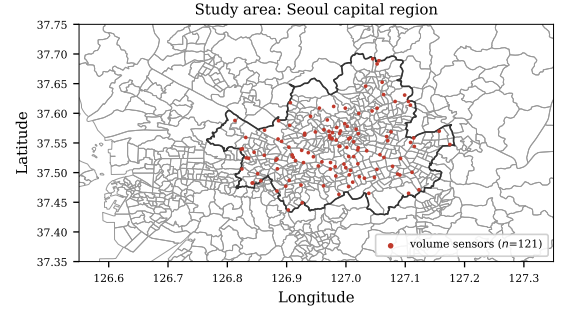


Fig. 2. Study area: Seoul (bold outline) and the surrounding capital region. Thin lines are the 500 OD administrative units (Section 3); red dots are the 121 volume sensors, all within Seoul proper. The 396 speed sensors (link geometries rather than points) are distributed similarly across the same road network and are omitted here for legibility.

TABLE 3
Training hyperparameters. Shared values apply to all models unless overridden.

Hyperparameter	Value
Input / forecast length P, Q	12 / 12 hours
Hidden size	64 (baselines); 48 default (ours)
Optimizer	Adam, $\text{lr} \times 10^{-3}$
Batch size	32 (baselines); 64 (ours)
Max epochs / patience	40 / 8 (15 / 5 for the mechanism-(iv) screen)
Training loss	smooth- L_1
DCRNN diffusion steps	2 (bidirectional)
Graph WaveNet dilation layers	4, kernel size 2
GTS graph-learning temperature	Gumbel-softmax $\tau = 0.5$
k NN graph (ours)	$k = 4$, Gaussian kernel, adaptive bandwidth
Routing hierarchy exponent α	0.9
Multipath perturbation (explored)	5 passes, log-normal $\sigma = 0.15$

(Figure 3). Volume MAPE barely moves (-0.06%) because MAPE weights low-volume off-peak hours as heavily as rush hours, whereas MSE is dominated by the high-volume errors where the gain concentrates.

Absolute performance vs. published architectures. Our base model is deliberately lightweight and does not match DCRNN or Graph WaveNet in absolute terms: at fold 0, Traffic-Only reaches 178,251 MSE and Traffic-Hybrid 157,937, against 128,014 for plain DCRNN. The claim is not that injection makes a small model state-of-the-art, but that a cheap auxiliary signal closes 40.4% of the gap to a considerably stronger architecture with no change in capacity – a partial substitute for capacity, most useful where capacity is constrained.

Capacity sweep. To see how the injection effect interacts with capacity, we retrained Traffic-Only/Hybrid at hidden sizes 64 and 96 on fold 0. Both improve with capacity (Only: 178,251 $\rightarrow$ 165,573 $\rightarrow$ 129,782; Hybrid: 157,937 $\rightarrow$ 147,060 $\rightarrow$ 127,629), and at hidden 96 Traffic-Hybrid matches plain DCRNN (-0.3%). But the injection effect itself does not stay constant: it is -11.4% at hidden

TABLE 4

Traffic-Only vs. Traffic-Hybrid (OD-routed injection), lightweight base model, 30-fold rolling-origin evaluation over the full 2023–2026 extended range (Section 3). Each metric's Δ is (Hybrid – Only)/Only $\times 100\%$, averaged over 30 folds; negative is an improvement. Wins counts folds where Hybrid beats Only on MSE.

Task	Wins	MSE Δ	RMSE Δ	MAE Δ	MAPE Δ
Volume	29/30	−7.17%	−3.69%	−4.02%	−0.06%
Speed	30/30	−8.17%	−4.18%	−4.10%	−3.65%

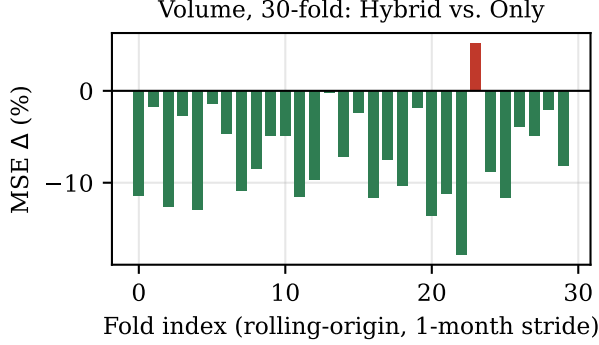


Fig. 3. Per-fold MSE change (Hybrid vs. Only), volume 30-fold set. Green = improvement, red = the single fold that regresses (fold 23, test month 2026-01); every other fold improves.

48, −11.2% at 64, and collapses to −1.7% at 96 (Figure 4). As the model gains enough capacity to match a far more sophisticated architecture on its own, the benefit of the routed signal shrinks toward zero – the first sign of the capacity gap that Section 7 establishes across architectures. This is a single-fold, single-seed trend; Section 7.4 repeats the test at full 30-fold resolution on a published architecture.

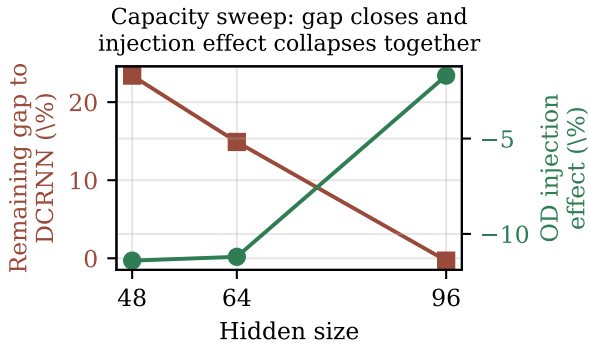


Fig. 4. Capacity sweep (hidden 48/64/96): the gap to DCRNN (red, left axis) closes and the OD-injection effect (green, right axis) collapses toward zero in lockstep as capacity increases.

Why the effect is larger for speed. The speed gain (−8.2%) consistently exceeds the volume gain (−7.2%) although both share the same OD signal, routing and fusion. Two explanations are plausible and not mutually exclusive. The speed network is three times denser and its sensors carry native link geometry (Section 3), so its routing matrix is better localized. And volume saturates at a road’s

physical capacity while speed degrades continuously under an unanticipated demand surge, leaving forward-looking demand information more headroom on speed. A sensor-density-matched comparison that could separate the two is not possible with Seoul’s deployed network.

6.1 Robustness Checks: Routing Necessity and Calendar Confounding

We test two threats to the headline result directly, holding architecture, fusion mechanism and training recipe fixed and varying only the injected signal, on the same 30-fold volume sweep (Table 5).

TABLE 5

Signal controls on the headline model (lightweight model, volume, 30 folds). Each row injects a different signal through the identical fusion mechanism; Δ is MSE change versus Traffic-Only. Recovery is each row’s Δ as a fraction of the real routed signal’s.

Injected signal	Wins	Mean Δ	p	Recovery
Real routed OD forecast (Hybrid)	29/30	−7.17%	8.0e-08	–
Unrouted OD forecast (no $\times W$)	20/30	−0.39%	0.61	5%
Calendar climatology of routed signal	27/30	−6.22%	2.9e-06	87%
Residual (routed – climatology)	24/30	−2.47%	0.019	34%
Climatology + residual, two channels	25/30	−3.88%	1.9e-04	54%

Is routing necessary? Removing the road-network projection $\times W$ leaves an OD signal indistinguishable from no injection (−0.39%, $p = 0.61$). The benefit is not produced by “any OD-shaped channel”: the routing step does real work.

Calendar confounding. The routed signal has a strong daily and weekly rhythm, so part of its benefit could come from the model learning a calendar prior rather than forward-looking OD content. For each fold we build a (day-of-week, hour-of-day) climatology of the routed signal from that fold’s *training window only* – a lookup table with no OD model and no forecast, which correlates with the real signal at $r = 0.997$. Injected alone, it recovers 87% of the gain: most of the average improvement comes from a calendar prior learned through the injection pathway.

Isolating the genuine OD content. Subtracting each fold’s climatology from the real signal, $\text{residual}_t = \text{real}_t - \text{climatology}[\text{bucket}(t)]$, leaves a signal with no calendar information by construction. Injected alone, it still recovers a significant 34% of the gain: day-specific OD content contributes, but as the secondary share. Feeding climatology and residual as two separately normalized channels lands in between (54%), so the natural quantity – real routed movement, climatology plus residual – is also the best one to inject.

All four controls use the 30-fold volume sweep; they have not yet been run on speed.

7 DOES INJECTION GENERALIZE TO STRONGER ARCHITECTURES?

A positive result on one lightweight architecture may not generalize. We therefore give the identical routed OD signal (Section 4.2) to the ten published architectures of Section 2.1, which span 31k–774k parameters – GMAN and STID are smaller than our own 62k-parameter model, the other eight are the same size or larger – and cover fixed-diffusion,

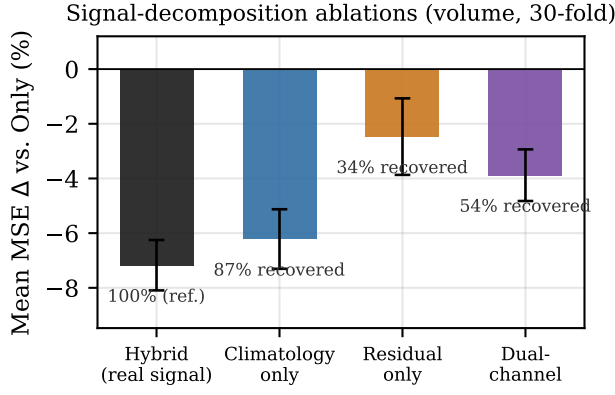


Fig. 5. The four signal-decomposition conditions (Hybrid, climatology-only, residual-only, dual-channel) plotted per fold, full 30-fold volume sweep. Climatology (blue) tracks Hybrid (black) closely across almost every fold; residual (orange) is smaller and noisier throughout, consistent with its modest 34% average recovery.

learned-adjacency, attention-based and graph-free designs. The main mechanism is the same gate our lightweight model uses; four alternatives test whether a failure could be blamed on the particular fusion design:

- (i) *Node-feature gated fusion* (main mechanism): the per-step gate of Section 4.3, applied to each architecture’s decoder representation, with an explicit future-calendar term.¹
- (ii) *Graph-adjacency modulation*: a per-node OD-activity gate rescales the architecture’s own fixed (DCRNN) or adaptive (Graph WaveNet, GTS) adjacency, injecting OD into *where information flows* rather than as a node feature.
- (iii) *Autoregressive decoding* (DCRNN): a true Q -step autoregressive decoder with the gate at every step, mirroring our own model exactly.
- (iv) *Boost/eliminate dual gate*: the knowledge-adaptation gate of [22], which up-weights hidden dimensions the auxiliary signal corroborates and suppresses those it contradicts – strictly more expressive than (i).
- (v) *Cross-attention* (STAEformer): each decode step attends over the OD signal at all Q future steps.

7.1 Uniform Injection Across Ten Architectures

The effect is ordered by capacity. Table 6 and Figure 6a–b show the main mechanism on all ten architectures, ordered by parameter count. Only the two smallest benefit significantly, both on volume only: STID (−9.02%, 30/30 folds, $p = 1.7 \times 10^{-10}$) and GMAN (−7.53%, 22/30, $p = 0.002$). STGCN and PDFormer, near 10^5 parameters, are null on volume. Every architecture above 10^5 parameters is significantly *harmful* on at least one task – ten of the twenty (architecture, task) combinations in total – and the harm is largest for the largest models (Graph WaveNet +25.45%, AGCRN +11.92%, MTGNN +11.07% on volume). Across

1. An early version without the calendar term was statistically indistinguishable from injecting temporally shuffled OD.

TABLE 6

OD injection (mechanism (i), node-feature gated fusion) across ten published or reimplemented architectures, ordered by parameter count. All rows are the extended 30-fold rolling-origin sweep. † = significant in the *favorable* (error-reducing) direction; all other bolded entries are significant and unfavorable. The two smallest architectures are the only ones that benefit; the effect turns null near 10^5 parameters and significantly harmful above it.

Model	Params	Wins	Volume Mean Δ (p)	Wins	Speed Mean Δ (p)
STID	31,028	30/30	−9.02%† (1.7e-10)	11/30	+0.24% (0.57)
GMAN	54,748	22/30	−7.53%† (0.002)	14/30	+0.19% (0.789)
STGCN	62,236	20/30	−2.09% (0.831)	16/30	+0.09% (0.925)
PDFormer	94,988	23/30	−2.70% (0.274)	2/30	+3.25% (1e-8)
STAEformer	159,996	14/30	+0.74% (0.484)	1/30	+4.71% (5e-10)
MTGNN	335,452	3/30	+11.07% (5e-5)	0/30	+4.24% (7e-14)
DCRNN	381,916	14/30	+0.95% (0.254)	11/30	+1.76% (0.011)
GTS	394,853	16/30	−0.19% (0.877)	3/30	+3.86% (5e-9)
GWNET	672,656	1/30	+25.45% (4e-9)	5/30	+3.77% (1e-5)
AGCRN	773,830	3/30	+11.92% (5e-6)	11/30	+0.91% (0.027)

all twenty points the effect correlates with $\log_{10}$ parameters at Spearman $\rho = 0.67$ ($p = 1.2 \times 10^{-3}$), and at $\rho = 0.89$ ($p = 5.4 \times 10^{-4}$) on volume alone. Speed is less cleanly ordered ($\rho = 0.45$): no small model benefits there, so capacity is a strong but not complete predictor, and we flag the volume/speed asymmetry as open. GMAN’s favorable volume result is not a lucky seed – seeds 0, 1 and 2 give −7.53%, −13.78% and −9.96%, all significant, and we report the most conservative.

It is not the fusion design. The alternative mechanisms show the same null-to-negative pattern. Graph-adjacency modulation on Graph WaveNet is null (+1.17%, 23 folds; Table 7). DCRNN’s autoregressive decoder is worse with OD than without it at fold 0 (140,689 vs. 134,693 MSE). The more expressive boost/eliminate gate, in a 6-fold warm-started screen, averages +2.32% (median −0.33%, two folds +6.7% and +10.1%). Cross-attention on STAEformer is +29.1% worse at fold 0, with visibly destabilized training, and was stopped there.

It is not forecast error. Injecting the *true*, already-observed OD instead of a forecast into DCRNN still gives no improvement (+1.31%, 4/9 folds; Table 7). Even a perfect OD forecast would not have helped, so a better forecaster is not the missing piece.

Why single-fold results mislead. At fold 0 alone, the main mechanism on STAEformer looked like a breakthrough (−3.67%), and a single-split study would have reported it as such. Over 30 folds it is noise (+0.74%, 14/30, $p = 0.48$) – a concrete example of the risk the rolling-origin protocol guards against.

TABLE 7

Supplementary mechanism diagnostics, intentionally not rerun at 30 folds (Section 3).

Model	Mech.	Task	Folds	Mean Δ (p)
DCRNN	oracle (true OD)	volume	9	+1.31% (−)
GWNET	(ii) graph-adj.	volume	23	+1.17% (0.522)

We call this pattern the *capacity gap*: once an architecture can infer from traffic history most of what the routed signal would tell it, a fixed fusion mechanism has little to gain

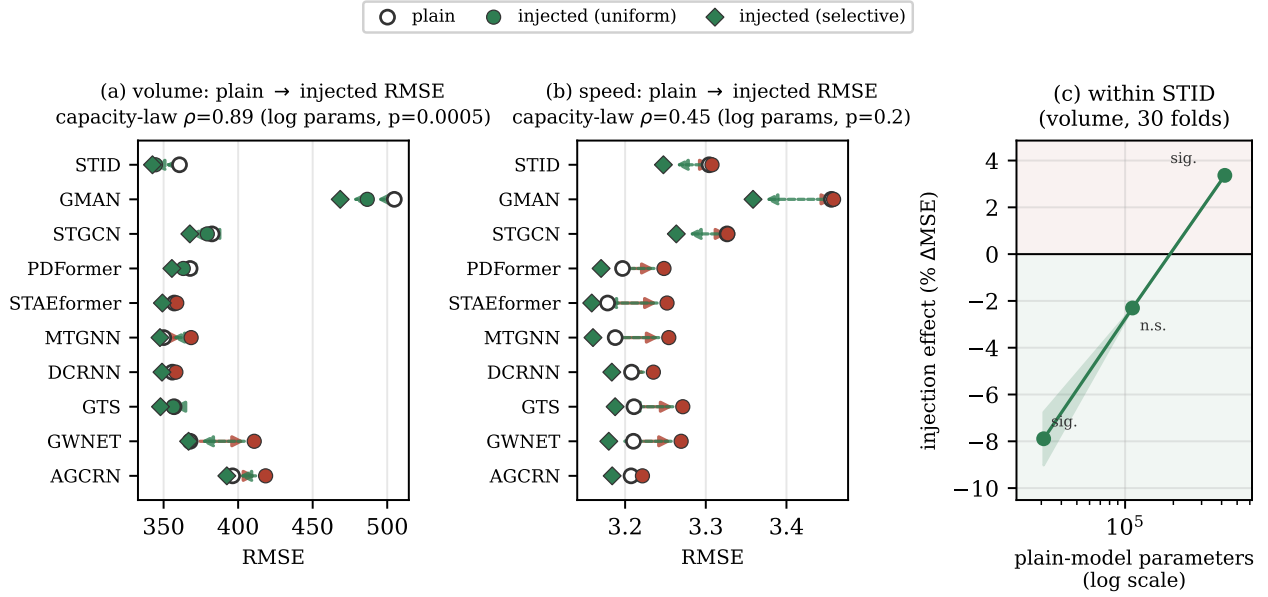


Fig. 6. Injection effect versus model capacity, shown as absolute RMSE moving through a three-point chain: plain (open circle) $\rightarrow$ uniformly injected (filled circle, colored by direction: green improves, red worsens) $\rightarrow$ selectively injected (filled diamond, this paper’s practical recipe, Section 7.2). (a) Volume and (b) speed, all ten architectures, 30-fold, ordered by parameter count (smallest at top): the small architectures at the top of each panel move left under uniform injection already, while the large architectures at the bottom move right (worsen) under uniform injection but the diamond recovers most of them back to the left of plain, often past it – the ordering of the uniform-injection step correlates with $\log_{10}$ parameters at Spearman $\rho = 0.89$ (volume, $p = 5.4 \times 10^{-4}$) and $\rho = 0.45$ (speed, $p = 0.2$); pooled across both tasks, $\rho = 0.67$ ($p = 1.2 \times 10^{-3}$). (c) The same uniform-injection decay reproduced *within* STID by varying only hidden size (30 folds per point; the band spans the two seeds run at hidden 64), with architecture, training recipe and injected signal held fixed.

and enough spare capacity to fit the signal’s imperfections. Sections 7.4–7.5 test this account directly. First, however, we ask whether the signal’s value is really gone in these models, or only inaccessible to their fusion mechanism.

7.2 Selective, Validation-Gated Injection

Table 6 shows that a fixed fusion mechanism fails to extract net value *in aggregate*; it does not show that no sensor benefits. The gate β_t is in principle a per-node trust decision, but it is trained only through one aggregate loss over all sensors and steps, with no direct signal about *which* sensors should be shielded from the OD branch – and in the larger architectures it evidently fails to learn this.

The selector. We replace this learned, soft decision with an explicit, hard one made outside the model. For each fold and each (architecture, task), we already have two trained checkpoints: plain and injected (mechanism i). We compute every sensor’s MSE for both checkpoints on that fold’s *validation* month and assign each sensor to whichever checkpoint was better there. This per-sensor choice is then frozen and applied to the fold’s test month. No test data enters the choice, no model is retrained, and the only extra cost is one validation-window forward pass per checkpoint.

Results. Table 8 compares selective with uniform injection on all twenty combinations (the five architectures tested first; STGCN and STID, added to probe whether Graph WaveNet/volume’s null result generalizes; and PDFormer, MTGNN and AGCRN, added because uniform injection harmed them most). Selective injection is significantly favorable in **eighteen of twenty** combinations. It recovers

eight of the ten combinations where uniform injection was significantly harmful – including AGCRN/volume (+11.92% $\rightarrow$ -1.86%) and MTGNN/speed (+4.24% $\rightarrow$ -1.71%) – and it strengthens both combinations that uniform injection already helped (GMAN/volume -7.53% $\rightarrow$ -14.14%; STID/volume -9.02% $\rightarrow$ -10.00%). The two exceptions are both volume combinations that uniform injection harmed severely: Graph WaveNet (-0.46%, $p = 0.36$), where uniform injection was catastrophic (+25.45%) and the selector chose injection for only 43 of 121 sensors on average; and MTGNN (-1.37%, $p = 0.074$), directionally favorable but short of significance. Both remain favorable in direction; neither becomes harmful.

What this means in absolute accuracy. Table 9 reports every model’s absolute RMSE before and after selective injection, including our own lightweight model. All eleven models improve on both tasks. On our own model, where uniform injection is already favorable, the raw selective gain is larger (-9.41% volume, -9.23% speed, vs. -7.17% and -8.17%), but Section 7.3 shows this extra margin is within the selector’s own noise baseline. The best results in the study are both selective: STID on volume (RMSE 360.7 $\rightarrow$ 342.5) and STAEformer on speed (3.18 $\rightarrow$ 3.16), each better than any plain model. Among plain models, MTGNN is best on volume RMSE and STAEformer on speed; our lightweight model trails all ten architectures except GMAN on volume, as intended of a deliberately small baseline.

TABLE 8

Selective (validation-gated, per-sensor) vs. uniform node-feature injection, all twenty (architecture, task) combinations, both columns on the same extended 30-fold sweep as Table 6. Bold = significant (favorable for Selective, either direction for Uniform). STGCN and STID (first rule) were added to test whether GWNET/volume’s null result generalizes to other low-signal combinations; it does not. PDFormer, MTGNN, and AGCRN (second rule) were added to test whether selective injection recovers the three architectures most harmed by uniform injection (Table 6); it does, in five of six combinations.

Model	Task	Selective Δ (wins, p)	Uniform Δ (wins, p)
DCRNN	volume	−4.06% (24/30, 0.0013)	+0.95% (14/30, 0.254)
DCRNN	speed	−1.51% (26/30, 6.2e-7)	+1.76% (11/30, 0.011)
GWNET	volume	−0.46% (16/30, 0.356)	+25.45% (1/30, 4.0e-9)
GWNET	speed	−1.87% (29/30, 1.6e-5)	+3.77% (5/30, 1.2e-5)
GTS	volume	−5.10% (27/30, 1.1e-6)	−0.19% (16/30, 0.877)
GTS	speed	−1.44% (28/30, 6.6e-10)	+3.86% (3/30, 4.8e-9)
GMAN	volume	−14.14% (27/30, 5.3e-10)	−7.53% (22/30, 0.0020)
GMAN	speed	−5.54% (30/30, 9.6e-19)	+0.19% (14/30, 0.789)
STAEformer	volume	−4.49% (23/30, 0.0009)	+0.74% (14/30, 0.484)
STAEformer	speed	−1.23% (26/30, 1.7e-6)	+4.71% (1/30, 4.6e-10)
STGCN	volume	−7.84% (27/30, 4.9e-6)	−2.09% (20/30, 0.831)
STGCN	speed	−3.73% (30/30, 4.4e-14)	+0.09% (16/30, 0.925)
STID	volume	−10.00% (30/30, 9.0e-14)	−9.02% (30/30, 1.7e-10)
STID	speed	−3.39% (30/30, 3.5e-13)	+0.24% (11/30, 0.567)
PDFormer	volume	−6.73% (29/30, 5.3e-7)	−2.70% (23/30, 0.274)
PDFormer	speed	−1.65% (29/30, 1.2e-8)	+3.25% (2/30, 1.1e-8)
MTGNN	volume	−1.37% (22/30, 0.074)	+11.07% (3/30, 5.5e-5)
MTGNN	speed	−1.71% (30/30, 2.2e-12)	+4.24% (0/30, 7.0e-14)
AGCRN	volume	−1.86% (22/30, 0.0063)	+11.92% (3/30, 4.7e-6)
AGCRN	speed	−1.45% (27/30, 8.1e-10)	+0.91% (11/30, 0.027)

TABLE 9

Absolute performance before and after selective injection, 30-fold mean, all eleven models ordered by parameter count. RMSE (inj.) uses the validation-gated selector of Section 7.2 for every model, including our own. MAPE and r are for the plain model. Bold marks the best value per column.

Model	Params	Volume				Speed			
		RMSE (plain)	RMSE (inj.)	MAPE	r	RMSE (plain)	RMSE (inj.)	MAPE	r
STID	31–35k	360.7	342.5	15.68%	0.988	3.30	3.25	9.61%	0.969
GMAN	55–59k	504.8	468.5	24.99%	0.976	3.46	3.36	10.18%	0.966
Ours (lightweight)	62k	410.6	390.7	17.95%	0.984	4.03	3.84	12.50%	0.953
STGCN	62–67k	382.5	367.6	16.52%	0.986	3.33	3.26	9.61%	0.968
PDFormer	95–157k	367.8	355.6	14.70%	0.987	3.20	3.17	9.23%	0.971
STAEformer	160–217k	357.1	349.2	13.96%	0.988	3.18	3.16	9.07%	0.971
MTGNN	335–349k	350.1	347.6	14.64%	0.988	3.19	3.16	9.20%	0.971
DCRNN	382–386k	355.8	348.9	15.17%	0.988	3.21	3.18	9.27%	0.971
GTS	395–422k	357.0	347.9	15.38%	0.988	3.21	3.19	9.31%	0.971
GWNET	673–683k	367.8	366.8	15.99%	0.987	3.21	3.18	9.20%	0.971
AGCRN	774–777k	396.3	392.4	16.86%	0.985	3.21	3.18	9.16%	0.971

7.3 Is the Selector Itself a Free Lunch?

Per-sensor validation MSE over one month is noisy. Choosing, per sensor, whichever of *any* two checkpoints looks better on a noisy estimate can produce an apparent gain even if neither carries a real advantage – the selection procedure, not the injected signal, would then be doing the work. (A random-selection control, supplementary Appendix B, rules out generic checkpoint-mixing but not this.) We measure this “free lunch” directly: for each architecture we train a second *plain* checkpoint differing only in random seed and apply the identical selector between the two plain checkpoints. Neither carries the OD signal, so whatever gain this produces is pure selection noise, and we subtract it from the raw selective result (Table 10). The free lunch must be measured per combination, because it grows with how much two seeds of an architecture disagree (seed-to-seed swing ranges from 1.4% to 9.7%).

Net of its own free lunch, selective injection remains **significantly favorable in eleven of twenty** combinations

(Benjamini–Hochberg $q < 0.05$), and favorable but not significant in six more. The significant rows include every strong result – GMAN/volume (−14.14% → −9.98% net), STID/volume (−10.00% → −8.52%), STGCN/volume (−7.84% → −5.40%) – and, among the architectures uniform injection harmed, PDFormer and MTGNN on speed. The six non-significant rows are almost all speed tasks on larger architectures, where the raw selective gain is small to begin with and mostly consists of selection noise.

Three combinations reverse sign – their raw gain is smaller than their free lunch: Graph WaveNet, MTGNN and AGCRN, all on volume (significantly so only for Graph WaveNet, $p = 0.004$). They are not scattered exceptions. They are exactly the three combinations uniform injection harmed most in the entire study (+25.45%, +11.07%, +11.92%), where the injected checkpoint is so much worse that the selector has almost no genuine signal to exploit. AGCRN/speed, whose uniform harm was mild (+0.91%), stays favorable (−0.27% net). The selector therefore fails

TABLE 10

Free-lunch-corrected selective injection: net effect after subtracting each architecture's own seed-noise-only selection baseline, all twenty (architecture, task) combinations in Table 8, sorted by net effect, plus our own lightweight model. p is a paired t -test of the per-fold net effect against zero; bold marks Benjamini–Hochberg $q < 0.05$ across all 22 rows.

Model	Task	Raw Δ	Free lunch	Net Δ	p (net)
GMAN	volume	−14.14%	−4.16%	−9.98%	$1e-4$
STID	volume	−10.00%	−1.47%	−8.52%	$1e-4$
STGCN	volume	−7.84%	−2.43%	−5.40%	$5.4e-04$
GMAN	speed	−5.54%	−1.67%	−3.88%	$1e-4$
GTS	volume	−5.10%	−1.41%	−3.69%	$6.1e-04$
PDFormer	volume	−6.73%	−3.04%	−3.68%	0.001
STID	speed	−3.39%	−0.67%	−2.72%	$1e-4$
STGCN	speed	−3.73%	−1.11%	−2.62%	$1e-4$
DCRNN	volume	−4.06%	−1.49%	−2.57%	0.021
STAEformer	volume	−4.49%	−2.25%	−2.23%	0.115
PDFormer	speed	−1.65%	−0.78%	−0.87%	0.003
MTGNN	speed	−1.71%	−0.88%	−0.84%	$1e-4$
GWNET	speed	−1.87%	−1.41%	−0.47%	0.271
STAEformer	speed	−1.23%	−0.83%	−0.40%	0.141
DCRNN	speed	−1.51%	−1.21%	−0.30%	0.219
AGCRN	speed	−1.45%	−1.18%	−0.27%	0.130
GTS	speed	−1.44%	−1.24%	−0.20%	0.249
AGCRN	volume	−1.86%	−2.85%	+1.00%	0.356
MTGNN	volume	−1.37%	−2.92%	+1.55%	0.102
GWNET	volume	−0.46%	−3.68%	+3.22%	0.004
Ours	volume	−9.41%	−3.59%	−5.82%	$1e-4$
Ours	speed	−9.23%	−1.42%	−7.81%	$1e-4$

precisely where the capacity gap is widest, which is consistent with, not a challenge to, the capacity-gap account.

Our own lightweight model shows the complementary case (last rows of Table 10). Its selective gain survives the correction comfortably (−5.82% volume, −7.81% speed net), but it no longer exceeds uniform injection (−7.17%, −8.17%), which involves no selection and needs no correction. Where a model already benefits from uniform injection, per-sensor selection adds nothing beyond selection noise; its value lies in rescuing the larger architectures where uniform injection fails.

7.4 Within-Architecture Capacity Sweep on a Published Model

The between-architecture correlation is suggestive but confounded: architectures differ in far more than parameter count. The cleanest test varies capacity alone inside one published architecture. We use STID, which is the smallest architecture in the study, has the largest favorable effect, and – unlike GMAN – is already a healthy model at its default size (plain volume MSE 132,418 vs. GMAN's 260,733), so a collapse of the effect at higher capacity cannot be explained by a poorly trained small baseline finally converging. We retrain STID at hidden sizes 64, 128 and 256 (31k to 419k parameters), 30 folds each, with everything else fixed.

A $13\times$ increase in capacity carries the injection effect not just toward zero but through it (Table 11, Figure 6c): from −9.02% ($p = 1.7 \times 10^{-10}$) at hidden 64, to null at 128, to significantly *harmful* at 256 (+3.37%, $p = 0.014$) – the same sign reversal Table 6 shows between architectures, now produced by scaling a single one. Two seeds at hidden 64 both reach significance (−9.02%, −6.76%), so the curve's starting point is not a seed artifact. Meanwhile

TABLE 11

STID capacity sweep, volume, 30 folds per row, corrected architecture throughout. Injection's benefit decays monotonically with capacity, loses significance by hidden 128, and reverses to significantly harmful by hidden 256, while the plain model's own accuracy saturates.

Hidden	Params	Plain MSE	Injection Δ	p
64 (seed 0)	31,028	132,418	−9.02%	$1.7e-10$
64 (seed 1)	31,028	132,102	−6.76%	$1.4e-6$
128	111,196	124,190	−2.30%	0.347
256	418,988	122,712	+3.37%	0.014

the plain model's own accuracy improves and saturates (132,418 $\rightarrow$ 124,190 $\rightarrow$ 122,712): the signal stops helping, and starts hurting, exactly as the model stops needing it and gains the spare capacity to fit its imperfections. This is what a redundancy account predicts and an independent-information account does not. With the fold-0 sweep on our own model (Section 6), the pattern now holds in two model families.

7.5 Diagnostic: When Does the Signal Help?

Aggregating ~ 280 million per-sample predictions across DCRNN's 23-fold (native-range) volume sweep, we bin samples into quintiles by how far the injected OD value deviates from its own (day-of-week, hour-of-day) typical baseline – computed from the training window only – normalized per sensor and pooled across all 121 sensors and 23 folds, and measure mean error reduction (Only – Hybrid) per quintile. A decile-resolution repeat reproduces the same monotonic pattern within sampling noise.

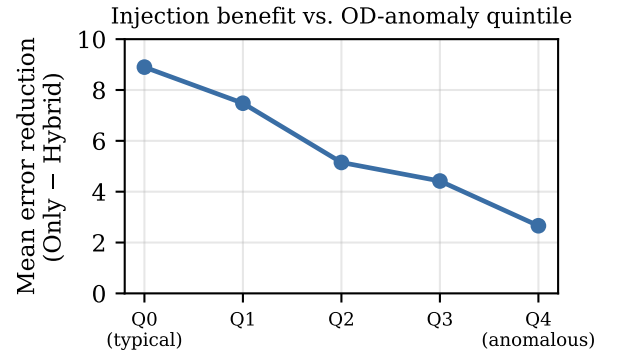


Fig. 7. Mean error reduction by OD-anomaly quintile: largest when the injected signal is closest to typical (Q0), smallest when it is most anomalous (Q4) – the opposite of the intuition that motivated this diagnostic.

Improvement is monotonically *largest* on the most typical samples and smallest on the most anomalous ones (Figure 7) – the opposite of the intuition that OD's value should lie in capturing surprises that past-traffic autocorrelation alone cannot. We read this as follows: the injected signal is itself a *forecast*, most accurate exactly when the pattern is typical. On typical hours it correctly confirms what a sufficiently strong model already infers – redundant but low-cost. On anomalous hours – the one regime where new information would matter most – the OD forecast is itself least reliable, so it cannot compensate for what autocorrelation misses.

A secondary check correlating each sensor's improvement with its own spatial autocorrelation (hypothesizing that injection should help most at spatially-hard-to-reason-about sensors) found only a weak positive association ($r = 0.161$; weak/mid/strong terciles improve $+4.51/+6.78/+5.98$), more plausibly explained by well-connected sensors having denser OD-routing coverage in W than by a capacity effect at the sensor level.

8 LIMITATIONS AND THREATS TO VALIDITY

Most of the lightweight-model gain is calendar-shaped. A training-window climatology of the routed signal recovers 87% of the uniform-injection gain, and the calendar-free residual alone recovers 34% (Section 6.1). Day-specific OD content contributes, but as the secondary share, and these controls have so far been run on volume only.

The selector's gain is only partly beyond selection noise. Net of the seed-control free lunch, the selective gain is significant in eleven of twenty combinations; six more, mostly speed tasks on larger architectures, are favorable but not significant, and three reverse (Section 7.3). On larger architectures, selective injection is best read as a reliable way to avoid the harm of uniform injection, with a genuine gain beyond that only in some combinations.

Fold non-independence. Adjacent folds share 11 of 12 training months, so the paired-test p -values are likely somewhat optimistic. We therefore read them together with the per-fold consistency of each result (e.g. win counts, Figure 3), not in isolation.

Static routing and a frozen forecaster. The routing matrix W is built once from 2023 destination shares, and the OD forecaster is frozen at a fold-0 checkpoint; neither is updated for later folds. Road topology changes little over this period, but shifts in destination preference and forecaster staleness could both understate the injection effect in later folds, and may contribute to the diagnostic of Section 7.5.

Scope. All main experiments use one city, one OD data source and one family of fusion mechanisms (gated or adjacency-level fusion of a routed scalar). The capacity law replicates in direction on three further public benchmarks with a calendar signal (supplementary Appendix C), but reaches significance on only one of them (PEMS-BAY); the smaller or noisier datasets (PeMSD7, METR-LA) do not. Mechanism pilots (iii)–(v) and the oracle were run on fewer folds, one of them (mechanism iv) with warm-started rather than independent per-fold training, and are reported as supporting rather than confirmatory evidence.

9 CONCLUSION

Forecast population movement, routed onto the road network, does carry information that traffic models can use – but whether a model benefits depends on how the signal is injected. Injected uniformly through a learned gate, routed OD improves a lightweight graph-recurrent model by 7.2% (volume) and 8.2% (speed) in MSE, closing 40% of its gap to DCRNN. The same mechanism does not generalize: across ten published architectures it helps only the two smallest and is significantly harmful on at least one task for every architecture above 10^5 parameters, in an order that tracks

model size and that scaling a single architecture reproduces. An oracle experiment and alternative fusion designs rule out forecast error and fusion expressiveness as the explanation; a per-sample diagnostic shows the signal is most reliable exactly where a strong model needs it least.

This capacity gap limits what an architecture's own fusion mechanism can learn, not the value in the signal. A per-sensor selector fitted on validation data alone, with no retraining, recovers that value: it improves all eleven models in absolute RMSE, is significant in eighteen of twenty architecture/task combinations – including eight of the ten where uniform injection was harmful – Net of a measured selection-noise baseline, the gain stays significant in eleven of twenty combinations and reverses only where uniform injection did the most damage; on our lightweight model, which uniform injection already helps, it adds nothing beyond that baseline.

For practitioners, the recommendation is simple: do not inject an auxiliary signal uniformly into a strong model; train the injected and plain variants and select per sensor on validation data. For researchers, two directions follow. The anomalous hours where current OD forecasts are least reliable (Figure 7) are where genuinely new information would matter most, which calls for event- or anomaly-conditioned injection rather than a plain point forecast. And the capacity-gap pattern should be tested in other upstream-cause/downstream-effect settings – weather in energy demand, flight schedules in airport ground traffic – to establish whether it is specific to OD-to-traffic injection or a general property of auxiliary-signal injection.

REFERENCES

- [1] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2018.
- [2] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *International Conference on Learning Representations (ICLR)*, 2018.
- [3] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph wavenet for deep spatial-temporal graph modeling," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.
- [4] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting," in *AAAI Conference on Artificial Intelligence*, 2019.
- [5] C. Zheng, X. Fan, C. Wang, and J. Qi, "Gman: A graph multi-attention network for traffic prediction," in *AAAI Conference on Artificial Intelligence*, 2020.
- [6] H. Liu, Z. Dong, R. Jiang, J. Deng, J. Deng, Q. Chen, and X. Song, "Staeformer: Spatio-temporal adaptive embedding makes vanilla transformer sota for traffic forecasting," in *ACM International Conference on Information and Knowledge Management (CIKM)*, 2023.
- [7] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [8] C. Shang, J. Chen, and J. Bi, "Discrete graph structure learning for forecasting multiple time series," in *International Conference on Learning Representations (ICLR)*, 2021.
- [9] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2020.
- [10] S. Han, J. An, and D. Lee, "Spatio-temporal road traffic prediction using real-time regional knowledge," *arXiv preprint arXiv:2408.12882*, 2024.
- [11] Z. Shao, Z. Zhang, F. Wang, W. Wei, and Y. Xu, "Spatial-temporal identity: A simple yet effective baseline for multivariate time series forecasting," in *ACM International Conference on Information and Knowledge Management (CIKM)*, 2022, pp. 4454–4458.

- [12] J. Jiang, C. Han, W. X. Zhao, and J. Wang, "Pdformer: Propagating delay-aware dynamic long-range transformer for traffic flow prediction," in *AAAI Conference on Artificial Intelligence*, 2023.
- [13] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. Holden-Day, 1970.
- [14] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [15] S. Guo, Y. Lin, H. Wan, X. Li, and G. Cong, "Learning dynamics and heterogeneity of spatial-temporal graph data for traffic forecasting," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 11, pp. 5415–5428, 2022.
- [16] C. Song, Y. Lin, S. Guo, and H. Wan, "Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting," in *AAAI Conference on Artificial Intelligence*, 2020.
- [17] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, and H. Li, "T-gcn: A temporal graph convolutional network for traffic prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 9, pp. 3848–3858, 2020.
- [18] J. Zhang, Y. Zheng, and D. Qi, "Deep spatio-temporal residual networks for citywide crowd flows prediction," in *AAAI Conference on Artificial Intelligence*, 2017.
- [19] Z. Shao, Z. Zhang, F. Wang, W. Wei, and Y. Xu, "Decoupled dynamic spatial-temporal graph neural network for traffic forecasting," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, 2022.
- [20] J. Han, W. Zhang, H. Liu, T. Tao, N. Tan, and H. Xiong, "Bigst: Linear complexity spatio-temporal graph neural network for traffic forecasting on large-scale road networks," *Proceedings of the VLDB Endowment*, vol. 17, no. 5, pp. 1081–1090, 2024.
- [21] S. Han, J. An, and D. Lee, "Urban vibrancy embedding and application on traffic prediction," *arXiv preprint arXiv:2602.21232*, 2026.
- [22] C. Li, L. Bai, W. Liu, L. Yao, and S. T. Waller, "Knowledge adaption for demand prediction based on multi-task memory neural network," in *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, 2020, pp. 715–724.
- [23] L. Wang, X. Geng, X. Ma, F. Liu, and Q. Yang, "Cross-city transfer learning for deep spatio-temporal prediction," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.
- [24] Y. Jin, K. Chen, and Q. Yang, "Selective cross-city transfer learning for traffic prediction via source city region re-weighting," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022.
- [25] Y. Wei, Y. Zheng, and Q. Yang, "Transfer knowledge between cities," in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.
- [26] Y. Yuan, J. Ding, J. Feng, D. Jin, and Y. Li, "A universal pre-training and prompting framework for general urban spatio-temporal prediction," *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 5, pp. 2212–2225, 2025.
- [27] Z. Li, L. Xia, Y. Xu, and C. Huang, "Gpt-st: Generative pre-training of spatio-temporal graph neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [28] Z. Shao, Z. Zhang, F. Wang, and Y. Xu, "Pre-training enhanced spatial-temporal graph neural network for multivariate time series forecasting," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022.
- [29] Z. Li, L. Xia, J. Tang, Y. Xu, L. Shi, L. Xia, D. Yin, and C. Huang, "Urbangpt: Spatio-temporal large language models," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2024.
- [30] C. Liu, S. Yang, Q. Xu, Z. Li, C. Long, Z. Li, and R. Zhao, "Spatial-temporal large language model for traffic prediction," *arXiv preprint arXiv:2401.10134*, 2024.
- [31] Z. Li, L. Xia, L. Shi, Y. Xu, D. Yin, and C. Huang, "Opencity: Open spatio-temporal foundation models for traffic prediction," *arXiv preprint arXiv:2408.10269*, 2024.
- [32] J. G. Wardrop, "Some theoretical aspects of road traffic research," *Proceedings of the Institution of Civil Engineers*, vol. 1, no. 3, pp. 325–362, 1952.
- [33] M. Beckmann, C. B. McGuire, and C. B. Winsten, *Studies in the Economics of Transportation*. Yale University Press, 1956.
- [34] Y. Sheffi, *Urban Transportation Networks: Equilibrium Analysis with Mathematical Programming Methods*. Prentice-Hall, 1985.
- [35] R. B. Dial, "A probabilistic multipath traffic assignment model which obviates path enumeration," *Transportation Research*, vol. 5, no. 2, pp. 83–111, 1971.